

Lab 2 Advanced: FPGA Implementation of Subset-Sum

Submission Due Dates:

Demo: 2026/09/29 17:20
 Source Code: 2026/09/29 18:30
 Report: 2026/10/04 23:59

Objectives

- Get familiar with SystemVerilog **combinational and sequential behavior modeling**.
- Practice **switch and LED control** on the FPGA board.

Introduction to the Subset-Sum Problem

- Given non-negative integers and a target T , subset-sum asks whether at least one subset sums exactly to T .
- Example: values $\{3, 5, 9, 11\}$, $T = 14 \rightarrow$ satisfiable, e.g., $\{3, 11\}$ or $\{5, 9\}$.
- Example: values $\{2, 4, 6, 8\}$, $T = 5 \rightarrow$ unsatisfiable (all sums are even).
- The empty subset sums to 0, so **$T = 0$ is always satisfiable**.
- Subset-sum is NP-complete. This lab uses only four values, so your circuit can enumerate all **16 subsets** by brute force.
- This lab uses a **best-fit variant**: choose the largest subset sum that does not exceed T . An exact match may not exist. The empty subset has sum 0, so a valid best fit always exists.

For more details, see Wikipedia: [Subset sum problem](#)

Action Items

1. lab2_adv_1.sv (35%)

Design a **combinational brute-force solver** for four values, then register its result at the rising edge.

- Evaluate all $2^4 = 16$ masks, from $4'b0000$ through $4'b1111$, and choose the valid mask with the largest sum. Register the result at the next rising edge.
- If several masks have the same best sum, keep the **lowest mask (the first one tested)**. Set `exact` when the chosen sum equals target.

a. IO List and Specification

Signal	I/O	Width	Description
clk	input	1	100 MHz clock; outputs update on rising edges
rst	input	1	Synchronous active-high reset
val_1–val_4	input	4 each	Unsigned values
target	input	6	Unsigned target; maximum possible sum is 60
out	output	4	Best-fit subset mask; out[0] selects val_1
exact	output	1	1 when the best sum equals target; otherwise 0

Timing and selection rules

- `rst` is a **synchronous active-high reset**. If `rst` is 1 at a rising edge, set `out` and `exact` to 0; **reset has priority**.
- Inputs may update at the **negative clock edge**.
- Test masks in order from $4'b0000$ through $4'b1111$. A later mask replaces the current best only if its sum is **strictly greater**. This preserves the first mask on a tie.
- At each rising edge, register the result computed from the current inputs. Hold `out` and `exact` stable until the next rising edge.

b. Example

val_1	val_2	val_3	val_4	target	out (next posedge)	exact
3	5	9	11	14	4'b0110	1
3	5	9	11	0	4'b0000	1
2	4	6	8	5	4'b0010	0
1	2	3	4	63	4'b1111	0

- Row 1: masks 4'b0110 ({val_2, val_3}) and 4'b1001 ({val_1, val_4}) both sum to 14. The tie-break selects 4'b0110, the first mask in enumeration order; exact = 1.
- Row 3: no subset sums to 5; the best sum not exceeding 5 is 4 (mask 4'b0010), so exact = 0.
- Row 4: target exceeds the maximum possible sum, so the full set is the best fit (sum 10, mask 4'b1111), and exact = 0.

c. Required design template

```
`timescale 1ns/1ps
```

```
module lab2_adv_1 (
    input logic  clk,
    input logic  rst,
    input logic [3:0] val_1,
    input logic [3:0] val_2,
    input logic [3:0] val_3,
    input logic [3:0] val_4,
    input logic [5:0] target,
    output logic  exact,
    output logic [3:0] out
);
```

```
// Implement your design here.
```

```
endmodule
```

2. lab2_adv_1_t.sv (15%)

Write your own testbench (lab2_adv_1_t.sv) to verify lab2_adv_1.sv. In your report, explain how you built the testbench. It must cover all of the following:

- Run the simulation for **exactly 120 ns**.
- Assert **rst at 15 ns** and deassert it at **35 ns**.
- Use the eight input sets provided in the template, including at least one no-exact-match case, one tie, and one target = 0 case.
- Change test inputs only at **falling (negative) clock edges**.
- Check that out and exact change only at **rising (positive) clock edges**.

3. lab2_adv_2.sv (50%)

Implement the best-fit subset-sum solver on the FPGA. Enter four values and a target using switches; display the selected mask, its sum, and the exact-match flag on LEDs.

a. FPGA connections

Signal	Direction	FPGA connection	Purpose
rst	input	SW15	Asynchronous active-high reset
valid	input	SW14	Commits the prepared switch value
switch_inputtype	input	SW13–SW12	Selects which register(s) to load
switch_input	input	SW7–SW0	Value or target data
out	output	LD3–LD0	Best-fit subset mask; LD0 selects val_1
sum	output	LD9–LD4	6-bit best-fit sum
exact	output	LD15	1 when the best-fit sum equals target

Load selector

switch_inputtype	When valid = 1 at a rising edge
2'b00	Load val_1 from SW7–SW4 and val_2 from SW3–SW0
2'b01	Load val_3 from SW7–SW4 and val_4 from SW3–SW0
2'b10	Load target from SW5–SW0
2'b11	Reserved — load nothing

Behavior

- rst is an **asynchronous active-high reset**. When rst = 1, clear all stored values and target to 0. The solver then displays mask 0, sum 0, and exact = 1 for target 0.
- Use valid as a **load enable**: keep it at 0 while preparing switch_inputtype and switch_input, then hold it at 1 across at least one rising edge to commit. When valid is 0, switch changes must not alter stored values.
- Every stored value and target starts at 0. Therefore, immediately after reset, the empty subset is the exact best fit for target 0; **LD15 = 1 is expected**.
- out uses the same best-fit and tie-break rules as lab2_adv_1. sum is the selected subset sum. exact is 1 only when that sum equals target.

b. Example Operations

For steps 2–5, keep SW14 = 1'b0 while preparing the selector and input. Then hold SW14 = 1'b1 across a rising clock edge to store the value.

Step	Prepare and commit	Stored after edge	Expected LEDs / meaning
1	Reset SW15 = 1'b1, then 1'b0	Values = 0, 0, 0, 0 Target = 0	exact = 1'b1 mask = 4'b0000 sum = 6'b000000 Empty subset exactly matches target 0.
2	Load val_1, val_2 SW13–12 = 2'b00 SW7–0 = 8'b0011_0101 Set SW14=1 across a rising edge	val_1 = 3 val_2 = 5 Target = 0	exact = 1'b1 mask = 4'b0000 sum = 6'b000000 Target is still 0.
3	Load val_3, val_4 SW13–12 = 2'b01 SW7–0 = 8'b1001_1011 Set SW14=1 across a rising edge	val_3 = 9 val_4 = 11 Target = 0	exact = 1'b1 mask = 4'b0000 sum = 6'b000000 All values are stored; target remains 0.
4	Load target SW13–12 = 2'b10 SW5–0 = 6'b001110 Set SW14=1 across a rising edge	Target = 14	exact = 1'b1 mask = 4'b0110 sum = 6'b001110 5 + 9 = 14; 4'b0110 wins the tie with 4'b1001.

Step	Prepare and commit	Stored after edge	Expected LEDs / meaning
5	Change target SW13–12 = 2'b10 SW5–0 = 6'b111111 Set SW14=1 across a rising edge	Target = 63	exact = 1'b0 mask = 4'b1111 sum = 6'b011100 Full set is best: 28 < 63.

c. Required design template

```
`timescale 1ns/1ps

module lab2_adv_2 (
    input logic clk,
    input logic rst,
    input logic [7:0] switch_input,
    input logic [1:0] switch_inputtype,
    input logic valid,
    output logic [3:0] out,
    output logic [5:0] sum,
    output logic exact
);

    // Implement your design here.

endmodule
```

4. Questions & Discussion

Answer the following questions in your report.

1. What are the differences between blocking (=) and non-blocking (<=) assignments? Which should generally be used for combinational and sequential logic, respectively, and why?
2. The one-cycle brute force enumerates 2^4 masks. If the problem grows to 16 values, how would the hardware cost change? Propose a sequential alternative that tests one mask per cycle. How many cycles would it need, and what is the trade-off between the two designs?
3. In lab2_adv_2, what unexpected behavior might occur if the **valid** signal and the input registers are removed and the solver uses the switch inputs directly? Explain why.

5. Report Guidelines

Prepare your report using the provided Advanced Lab Report Template. In addition to the general requirements in the template, answer all questions in Section 4 and explain how you designed and verified your testbench, including appropriate simulation waveform(s).

Attention

- **DO NOT copy-and-paste code segments from the PDF materials to compose your design; invisible non-ASCII characters may cause hard-to-debug syntax errors.**
- **Hand in three source files: lab2_adv_1.sv, lab2_adv_1_t.sv, and lab2_adv_2.sv. Upload each file individually; DO NOT submit a compressed ZIP file.**
- Hand in your report as **lab2_adv_report_StudentID.pdf**.
- Be prepared to answer questions from the TA during the demo.
- Prepare the bitstream files before the lab demo so the demo proceeds smoothly.